

Practical – week 8 – 2520090235

Task1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // 1. malloc() - allocate memory for 3 integers
    int *arr = (int *)malloc(3 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    printf("Memory allocated using malloc():\n");
    for (i = 0; i < 3; i++) {
        printf("%d ", arr[i]);
    }

    // 2. calloc() - allocate memory for 3 integers
    // calloc initializes allocated memory to zero
    int *zeros = (int *)calloc(3, sizeof(int));

    if (zeros == NULL) {
        printf("\nCalloc allocation failed\n");
        free(arr);
        return 1;
    }

    printf("\n\nMemory allocated using calloc():\n");
    for (i = 0; i < 3; i++) {
        printf("%d ", zeros[i]);
    }

    // 3. realloc() - increase arr size from 3 to 5 integers
    int *temp = (int *)realloc(arr, 5 * sizeof(int));

    if (temp == NULL) {
        printf("\nRealloc allocation failed\n");
        free(arr);
        free(zeros);
        return 1;
    }

    arr = temp;

    arr[3] = 40;
    arr[4] = 50;

    printf("\n\nMemory after realloc():\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    // 4. free() - release allocated memory
    free(arr);
    free(zeros);
}
```

Output:

```

^C
rohita@LaxmiDurga:~$ nano p8.c
rohita@LaxmiDurga:~$ gcc p8.c
rohita@LaxmiDurga:~$ ./a.out
Memory allocated using malloc():
10 20 30

Memory allocated using calloc():
0 0 0

Memory after realloc():
10 20 30 40 50

All allocated memory has been released.

```

Observation: Observation: The program successfully demonstrates `malloc()`, `calloc()`, `realloc()`, and `free()`, and Valgrind can be used to verify that all dynamically allocated memory is released without memory leaks.

- `malloc()` allocates memory for 3 integers and stores 10, 20, 30.
- `calloc()` allocates memory for 3 integers and initializes them to 0.
- `realloc()` increases the first array from 3 integers to 5 integers and adds 40, 50.
- `free()` releases the allocated memory.

Task2:

Develop a program that demonstrates Copy-on-Write behaviour after a `fork()`. Analyse memory usage before and after modifying data in the child process.

Code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <string.h>

#define ARRAY_SIZE (20 * 1024 * 1024) // 20 MB

void print_memory(const char *message) {
    char path[64];

    snprintf(path, sizeof(path), "/proc/%d/status", getpid());

    FILE *file = fopen(path, "r");

    if (file == NULL) {
        perror("fopen");
        return;
    }

    char line[256];

    printf("\n%s (PID: %d)\n", message, getpid());

    while (fgets(line, sizeof(line), file)) {
        if (strncmp(line, "VmSize:", 7) == 0 ||
            strncmp(line, "VmRSS:", 6) == 0 ||
            strncmp(line, "RssAnon:", 8) == 0) {
            printf("%s", line);
        }
    }

    fclose(file);
}

int main() {
    setbuf(stdout, NULL);

    // Allocate 20 MB of memory
    char *data = malloc(ARRAY_SIZE);

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    // Touch every page so memory is physically allocated
    for (size_t i = 0; i < ARRAY_SIZE; i += 4096) {
        data[i] = 'A';
    }

    // Display memory before fork
    print_memory("Parent before fork()");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        // Child process
        print_memory("Child immediately after fork()");

        printf("\nChild is modifying the first half of the array...\n");

        // Modify half of the allocated memory
        for (size_t i = 0; i < ARRAY_SIZE / 2; i += 4096) {
            data[i] = 'B';
        }

        // Display memory after modification
        print_memory("Child after modifying memory");

        free(data);
        _exit(0);
    } else {
        // Parent process
        wait(NULL);

        print_memory("Parent after child exits");

        free(data);
    }

    return 0;
}

```

Output:

```

All allocated memory has been released.
rohita@LaxmiDurga:~$ nano p8.c
rohita@LaxmiDurga:~$ nano p81.c
rohita@LaxmiDurga:~$ gcc p81.c
rohita@LaxmiDurga:~$ ./a.out

Parent before fork() (PID: 1104)
VmSize:    23252 kB
VmRSS:     22300 kB
RssAnon:   20580 kB

Child immediately after fork() (PID: 1105)
VmSize:    23252 kB
VmRSS:     21484 kB
RssAnon:   20580 kB

Child is modifying the first half of the array...

Child after modifying memory (PID: 1105)
VmSize:    23252 kB
VmRSS:     21484 kB
RssAnon:   20580 kB

Parent after child exits (PID: 1104)
VmSize:    23252 kB
VmRSS:     22300 kB
RssAnon:   20580 kB

```

Observation: The program demonstrates Copy-on-Write after `fork()`, showing that the parent and child initially share physical memory pages, while modifying data in the child causes private copies of affected pages to be created.